Advanced Vision Systems

AGH UST International Course

Tomasz Kryjak, PhD, Mateusz Wąsala, MSc, Hubert Szolc, MSc, Piotr Wzorek, MSc, Krzysztof Błachut, MSc

Copyright © 2026
Tomasz Kryjak
Mateusz Wąsala
Hubert Szolc
Piotr Wzorek
Krzysztof Błachut

Embedded Vision Systems Group
Vision Systems Laboratory
WEAIiIB, AGH University of Krakow

Fourth edition, revised and corrected, Krakow, February 2026

Contents

I Blok 1

1 Laboratory 1

1 Vision algorithms in Python 3.X – introduction 9
1.1 Programming software 9
1.2 Python modules used in image processing 9
1.3 Input/output operations 10
1.4 Colour space conversion 10
1.4.1 OpenCV 10
1.4.2 Matplotlib 11
1.5 Scaling, rescaling with OpenCV 11
1.6 Arithmetic operations: addition, subtraction, multiplication, difference module 12
1.7 Histogram calculation 12
1.8 Histogram equalisation 13
1.9 Filtration 13

Blok 1

1 Laboratory 1

1 Vision algorithms in Python 3.X – introduction 9

1.1 Programming software
1.2 Python modules used in image processing
1.3 Input/output operations
1.4 Colour space conversion
1.5 Scaling, rescaling with OpenCV
1.6 Arithmetic operations: addition, subtraction, multiplication, difference module
1.7 Histogram calculation
1.8 Histogram equalisation
1.9 Filtration

1. Vision algorithms in Python 3.X – introduction

The exercise will present/mention basic information related to the use of the Python language to perform image and video processing operations and image analysis.

1.1 Programming software

Before starting the exercises, **please create** your own working directory in the place given by the tutor. The name of the directory should be associated with your name – the recommended form is `LastName_FirstName` without special characters.

The Python programming environment – Visual Studio Code (VSCode) – can be used to complete the exercises. Visual Studio Code is a free, lightweight, intuitive and easy to use code editor. It is a universal software for any programming language. Configuration of the editor for exercises comes down only to choosing the appropriate interpreter, or more precisely the appropriate virtual environment in the Python language.

1.2 Python modules used in image processing

Python provides built-in containers (e.g., lists), but it does not offer a native array type designed for efficient vectorised or element-wise operations (such as addition, multiplication, or thresholding of entire data blocks). In practice, the standard solution is to use the *NumPy* module, which provides the `ndarray` type as well as mechanisms such as vectorisation and *broadcasting*. A digital image can then be treated as an $H \times W$ array (grayscale) or $H \times W \times C$ array (color), usually of type `uint8` (range 0–255) or `float` (e.g., 0–1). *NumPy* forms the foundation for most libraries used later for image processing and visualisation.

The most commonly used packages supporting image processing include:

- *SciPy* (the *ndimage* module) – basic filtering and transformation operations (also multidimensional) and *Matplotlib* (*pyplot*) – visualisation of images and plots,
- *Pillow* (a fork of the older, no-longer-maintained PIL module) – convenient loading/saving and simple operations on images,

- *OpenCV* (the *cv2* module) – an extensive set of computer vision algorithms (filters, geometry, features, matching, calibration, etc.),
- *scikit-image* (*skimage*) – a library with image processing algorithms integrated with the Python scientific ecosystem,
- *Plotly* (*plotly*) – interactive plots and visualisations (e.g. heatmaps, 3D plots, dashboards in Jupyter/HTML),
- *Rerun* (*rerun*, *rerun.io*) – logging and interactive visualisation of data (e.g. images, annotations, 3D/robotics) in the Rerun viewer,
- *pandas* – tools for working with tabular data (e.g. loading annotations and metadata from CSV/Excel, analysis of experiment results).

In this course we will mainly rely on *OpenCV*. In parallel we will use *Matplotlib* (especially for displaying results; note: *OpenCV* by default stores colour in BGR order, while *Matplotlib* expects RGB) and occasionally *ndimage* – when a given operation is not available in *OpenCV* or is less convenient to use there.

1.3 Input/output operations

Reading, displaying and saving an image using OpenCV and Matplotlib

Exercise 1.1 Perform a task in which you practice handling files using OpenCV.

1. From the course page, download the image *mandril.jpg* and place it in your own working directory.
2. Load, display, and save an image with the .png extension under a new name using the OpenCV and Matplotlib libraries. Then place a **point** and a **rectangle** on the image and display it on the screen.

R Use:

- OpenCV: `cv2.imread()`, `cv2.imshow()`, `cv2.imwrite()`, `cv2.waitKey()`, `cv2.destroyAllWindows()`, `cv2.rectangle()`.
- Matplotlib: `plt.imread()`, `plt.imsave()`, `plt.figure()`, `plt.imshow()`, `plt.axis()`, `plt.show()` oraz `matplotlib.patches.Rectangle()`.

R The `waitKey()` function displays the image for a specified amount of time (the argument 0 means it waits for a key press). An unnecessary window can be closed with the `destroyWindow()` command with an appropriate parameter, and all open windows can be closed with the `destroyAllWindows()` command. As a good practice, you should always use the `cv2.destroyAllWindows()` function.

■

1.4 Colour space conversion

1.4.1 OpenCV

The function *cvtColor* is used to convert the colour space.

```
IG = cv2.cvtColor(I, cv2.COLOR_BGR2GRAY)
IHSV = cv2.cvtColor(I, cv2.COLOR_BGR2HSV)
```

R Please note that in OpenCV the reading of the image is in **BGR** order, not RGB. This can be important when manually manipulating pixels. For a full list of available

conversions with the relevant formulas check the OpenCV documentation.

Exercise 1.2 Convert the colour space of the given image.

1. Convert the *mandril.jpg* image to greyscale and HSV space. Display the result.
2. Display the H, S, V components of the image after conversion.

■

R Please note that in contrast to e.g. the Matlab package, here the indexing is from 0.

Useful syntax:

```
IH = IHSV[:,:,0]
IS = IHSV[:,:,1]
IV = IHSV[:,:,2]
```

1.4.2 Matplotlib

Here the choice of available conversions is quite limited. Therefore, we need to use the formulas for colour space conversion.

1. RGB to greyscale:

$$G = 0.299 \cdot R + 0.587 \cdot G + 0.144 \cdot B \tag{1.1}$$

The formula can be represented as a function:

```
def rgb2gray(I):
  return 0.299*I[:,:,0] + 0.587*I[:,:,1] + 0.114*I[:,:,2]
```

R The colour map must be set when displaying. Otherwise the image will be displayed in the default, which is not greyscale: `plt.gray()`

2. RGB to HSV.

```
import matplotlib # add at the top of the file
I_HSV = matplotlib.colors.rgb_to_hsv(I)
```

1.5 Scaling, rescaling with OpenCV

Exercise 1.3 Scale the image with *mandril*. Use the *resize* function to scale.

■

Example of use:

```
height, width = I.shape[:2] # retrieving elements 1 and 2, i.e. the corresponding
    height and width
scale = 1.75   # scale factor
Ix2 = cv2.resize(I,(int(scale*height),int(scale*width)))
cv2.imshow("Big Mandril",Ix2)
```

1.6 Arithmetic operations: addition, subtraction, multiplication, difference module

The images are matrices, so the arithmetic operations are quite simple – just like in the Matlab package. Of course, remember to convert to the appropriate data type. Usually double format will be a good choice.

Exercise 1.4 Perform arithmetic operations on the image *lena*.

1. Download the image *lena* from the course page, then load it using a function from OpenCV – add this code snippet to the file that contains the image loader *mandril*. Perform the conversion to greyscale. Add the matrices containing mandril and mena in greyscale. Display the result.
2. Similarly perform the subtraction and multiplication of images.
3. Implement a linear combination of images.
4. An important operation is the module from image difference. It can be done manually – conversion to the appropriate type, subtraction, module (`abs`), conversion to *uint8*. An alternative is to use the function *absdiff* from OpenCV. Please calculate the modulus from the difference of the mandril and lena greyscale images.

■

R When displayed, the image may not visualise correctly, because it is of type *float64* (*double*). In practice, you should:

1. limit the range of values (`clip`),
2. optionally rescale them to the interval `[0,255]`,
3. convert to *uint8*.

Appropriate conversion:

```
img8 = (255*np.clip(img,0,1)).astype(np.uint8) # when values are in [0,1] range
img8 = np.clip(img,0,255).astype(np.uint8) # when values are in [0,255] range
```

1.7 Histogram calculation

Calculating a histogram can be done using the `calcHist` function. Before we move on to that, let us recall the basic control structures in Python: functions and subroutines. Please complete on your own the following function, which computes the histogram of an image in 256 shades of gray:

```
def hist(img):
  h=np.zeros((256,1), np.float32) # creates and zeros single-column arrays
  height, width = img.shape[:2] # shape - we take the first 2 values
    for y in range(height):
...
return h
```

A histogram can be displayed using the `plt.hist` or `plt.plot` functions from the *Matplotlib* library.

The `calcHist` function makes it possible to compute a histogram for several images (or their channels), which is why it takes arrays (e.g. of images) as parameters rather than a single image. However, the following form is used most often:

```
hist = cv2.calcHist([IG],[0],None,[256],[0,256])
# [IG] -- input image
# [0] -- for greyscale images there is only one channel
```

```
# None -- mask (you can count the histogram of a selected part of the image)
# [256] -- number of histogram bins
# [0, 256] -- the range over which the histogram is calculated
```

Please check that the histograms obtained by both methods are the same.

1.8 Histogram equalisation

Histogram equalisation is a popular and important pre-processing operation.

1. Classical histogram equalisation is a global method, it is performed on the whole image. There is a ready-to-use function for this type of equalisation in the OpenCV library:

```
IGE = cv2.equalizeHist(IG)
```

2. CLAHE equalisation (Contrast Limited Adaptive Histogram Equalization) – an adaptive method that improves the lighting conditions in an image. Unlike classical histogram equalisation, it operates locally: it equalises the histogram in individual regions of the image rather than for the entire image. The method works as follows:
   - dividing the image into disjoint (square) blocks,
   - computing the histogram in each block,
   - equalising the histogram with a limit on the maximum "height" (the excess is redistributed to neighbouring bins),
   - interpolating pixel values based on the histograms of neighbouring blocks (usually taking into account the four nearest neighbours).

   Details can be found on Wikipedia and in the OpenCV tutorial.

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
# clipLimit - maximum height of the histogram bar - values above are distributed
    among neighbours
# tileGridSize - size of a single image block (local method, operates on separate
    image blocks)
I_CLAHE = clahe.apply(IG)
```

Exercise 1.5 Run and compare both equalisation methods. ■

1.9 Filtration

Filtering is a very important group of operations on images. As an exercise, please run:

- Gaussian filtration (`GaussianBlur`)
- Sobel filtration (`Sobel`)
- Laplacian filter (`Laplacian`)
- Median filtration (`medianBlur`)

R OpenCV documentation – filtration.

Please also note the other functions available:

- bilateral filtration,
- Gabor filters,
- morphological operations.

Exercise 1.6 Run and compare the methods mentioned.

Exercise completion

In order to have the exercise accepted, after completing all the tasks in this laboratory, report your readiness to the teacher. Once it has been approved by the teacher, place the script with the `.py` or `.ipynb` extension in the appropriate location in the course resources on UPeL. ■